WRAPPER CLASS

Wrapper classes helps the user to store all the primitive literals in the form of objects.

In java, we have built-in classes for each & every primitive data type which helps us to achieve the above task. Therefore, with the help of wrapper classes, we can convert a primitive type into a non-primitive type & vice versa.

All the wrapper classes are available inside a package called as lang package. Therefore, we can use them without even importing.

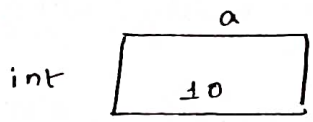
▶ LIST OF WRAPPER CLASSES :-

SL NO	PRIMITIVE DATA TYPE	NON-PRIMITIVE DATA TYPE
1.	byte	Byte.
2	short	Short
3.	int	Integer
4	long	Long
5	float	Float
6.	double	Double
7.	char	Character
8	boolean	Boolean.

Ex :

PRIMITIVE

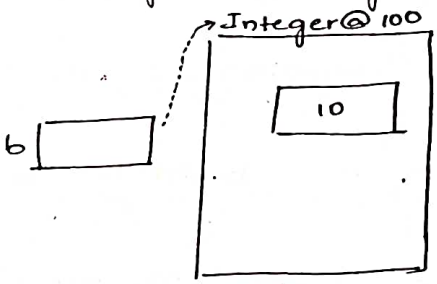
```
int a = 10;
```




```
S.o.pln(a); //10
So.pln(a.tostring()); //CTE
```

NON-PRIMITIVE

```
Integer b = Integer.valueOf(10
```




```
S.o.pln(b); //10
S.o.pln(b.tostring()); //10
```

```
Integer b = a;
        ↖ (int)
or
Integer b
= Integer.valueOf(a);
```

Boxing is done imp... from JDK version. Also conv...

can be done implicitly!

NOTE : In all the wrapper classes, the object class methods like tostring method, equals method & hashcode method are overrid...

2. All wrapper classes implements following interfaces:

   a. Comparable. ✓

   b. Serializable ✓

3. wrapper classes helps us to convert primitive type into non-primitive type. (BOXING)

4. wrapper classes helps us to convert non-primitive type in... primitive type. (UNBOXING)

▶ BOXING:

The process of converting primitive type into non-primitive type is known as Boxing.

From jdk 1.5 onwards, boxing is done implicitly, hence, it is also known as AUTOBOXING.

NOTE: Programmers can also do boxing explicitly with the help of an built-in method called valueOf() present inside each wrapper class.

```
Integer a = Integer.valueOf(10); // Explicit boxing
Integer a = 10; // Auto boxing.
```

BOXING METHODS:

PRIMITIVE TYPE	WRAPPER CLASS	RETURN TYPE	METHOD CALLING.
byte	Byte	Byte	Byte.valueOf(byte value);
short	Short	Short	Short.valueOf(short value)
int	Integer	Integer	Integer.valueOf(int value)
long	Long	Long	Long.valueOf(long value)
float	Float	Float	Float.valueOf(float value)
double	Double	Double	Double.valueOf(double value)
char	Character	Character	Character.valueOf(char value)
boolean	Boolean	Boolean	Boolean.valueOf(boolean value)
	String	String	String.valueOf(string value)

UNBOXING :-

The process of converting non-primitive data type into primitive data type is known as Unboxing.

From jdk 1.5 onwards, the process of Unboxing is done implicitly by the compiler, hence it is known as Auto Unbox

Ex: Integer a=10

int b=a; // Integer → int (auto unboxing)

NOTE: A programmer can perform unboxing explicitly using a non-static method present in respective Wrapper classes.

NON-PRIMITIVE TYPE	PRIMITIVE TYPE	RETURN TYPE	METHOD & ITS CALLING
Byte	byte	byte	ref.byteValue()
Short	short	short	ref.shortValue()
Integer	int	int	ref.intValue()
Long	long	long	ref.longValue()
Float	float	float	ref.floatValue()
Double	double	double	ref.doubleValue()
Character	char	char	ref.charValue()
Boolean	boolean	boolean	ref.booleanValue()

NOTE: In Java, as we can convert all the primitive types into its respective, non-primitive types & object class being the supermost class of all the non primitive data type. We can conclude that inside object class containers, we can store whatever we want.

Ex: class Employee
{
}
class Book
{
}
class Mis
{
public static void main (String [] args)
{
Object o;
o=10; // Autoboxing & Auto upcasting
System.out.println (o);
~~Object~~ o=20.0;
System.out.println (o);
o= new Scanner (System.in);
o= new Employee ();
}
}

I
Integer ← int
Autoboxing

II
Object ← Integer
Autoupcasting.

STRING TO PRIMITIVE TYPE CONVERSION:

In java, we can covert a string into its respective primitive data type. This process is known as "Parsing".

We can parse a string into its respective primitive data types, with the help of a static methods which is present inside each & every wrapper class as shown in the table below.

NON PRIMITIVE DATA TYPE	PRIMITIVE TYPE	RETURN TYPE	STATIC METHOD & ITS CALLIN
String	byte	byte	Byte.parseByte (String)
String	short	short	Short.parseShort (String)
String	int	int	Integer.parseInt (String)
String	long	long	Long.parseLong (String)
String	float	float	Float.parseFloat (String)
String	double	double	Double.parseDouble (String)
String	char	char	Character.parseChar (String)
String	boolean	boolean	Boolean.parseBoolean (Strin

Ex :

```
class W16
{
    public static void main (String[] args)
    {
     //String to int
     String s = "10";
     int a = Integer.parseInt(s);
     System.out.println(a);
    }
}
```

NOTE : NumberFormatException

Whenever we try to parse a string, in the passed string, if the passed string consists of any character other than digits / Numbers, we get NumberFormatException

COMMAND LINE ARGUMENTS :

In java we can pass inputs to the application in the following ways:

1) Directly within the application using hardcoding.

2) Using Scanner class.

3) Using command line Arguments.

USING COMMANDLINE ARGUMENTS:

The values which we pass during the execution comman of a program is known as Command Line Arguments.

Ex: java P1 10 20

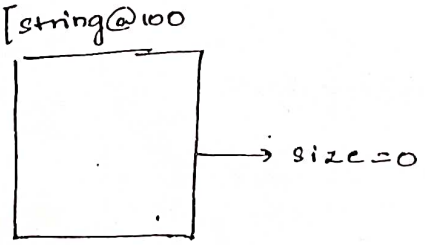
10 & 20 are command line arguments.

Where does the command line arguments will have the memory allocated to them.:

They will have the memories allocated to them, inside an array object which is created by JVM based on the number of arguments passed in the execution command.

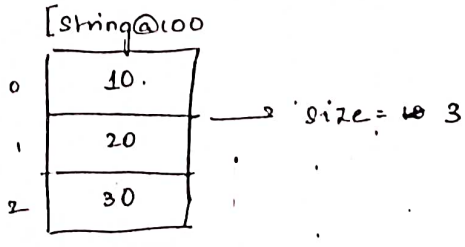
Ex: 1) If No i/p is passed then JVM will create a String array object of size 0 [zero]

Ex: java P1



2) If 'n' no of i/p is passed, the JVM will create String object with the size equals to 'n'

Ex: java P1 10 20 30



Eg: class P2

```
{
    public static void main (String[] args)
    {
     System.out.println("Main Begin");
     System.out.println("Ref variable: "+ args);
     System.out.println("Size: "+ args.length);
     System.out.println(args[0]);
     System.out.println(args[1]);
     System.out.println(args[2]);
    }
}
```

Op:

→ javac P2 10 20 ↵ Enter:

Main Begin

Ref variable: [String@100

Size: 2

10

20

ArrayIndexOutOfBoundsException.

→ java P2 10 20 30 ↵ enter

Main Begin

Ref variable: [String@100

Size: 3

10

20

30

WAJP to read your Best friend names & print the number of characters in each name along with it

```
class Read
{
  public static void main (String [] args)
  {
    ~~System.out.println (~~
    ~~for (int i = args.length~~
    for (int i=0; i< args.length; i++)
    {
      System.out.println ("Name"+(i+1)+" : "+ args[i]);
      System.out.println ("Number of characters :"+ args[i].length(
    }
  }
}
```